

献血管理系统性能优化报告

1. 报告目的

本报告说明系统在数据访问、索引、事务、日志写入和安全控制方面采用的措施，并记录可重复执行的压力测试结果。

课程要求的重点包括：

- MySQL 索引与 SQL 查询优化。
- JDBC 连接池和预编译语句。
- 核心写入流程的事务一致性。
- MongoDB 索引和聚合查询。
- 10000 条日志、50 并发的压力测试。
- 密码保护、SQL 注入防护、权限控制和测试数据安全。

2. 运行环境

组件	版本或环境
操作系统	macOS 15.7.2, ARM64
项目编译目标	Java 17
Maven	3.9.16
MySQL	8.0.46, Docker
MongoDB	5.0.33, Docker
MySQL 容器	hzcu-mysql
MongoDB 容器	hzcu-mongo

2.1 演示数据规模

执行计划分析使用当前 Docker 数据库中的演示数据。

MySQL 数据量：

表	记录数
users	10
categories	10

表	记录数
items	20
orders	25
profiles	10

MongoDB 数据量：

集合	文档数
action_logs	100
comments	23
item_details	20
system_logs	103

演示数据量较小，执行耗时主要用于发现明显错误，不用于证明 大数据量下的绝对性能。本报告同时检查索引选择、扫描行数、临时表、 额外排序和并发写入数量，避免只依赖毫秒级耗时下结论。

2.2 评估方法

评估对象	方法	关注指标
MySQL 查询	EXPLAIN	访问类型、命中索引、预计扫描行数、额外操作
MySQL 实际执行	EXPLAIN ANALYZE	实际行数、节点耗时、排序和临时表
MongoDB 查询	explain("executionStats")	IXSCAN 、检查键数、检查文档数
并发写	JUnit 压力测试	成功写入数、耗时、吞吐量、数据清理
事务	代码与自动化测试	锁定范围、提交、回滚、状态校验

2.3 优化目标

- 高频等值查询使用唯一索引或普通索引定位。
- 日志和评论列表的过滤字段与排序字段使用组合索引。
- 月度报表使用时间范围索引，不通过函数包装时间字段。
- 界面不加载无上限的日志和评论数据。
- 库存扣减使用短事务和行级锁，不锁定无关记录。
- 压力测试完成 10000 条日志、50 并发写入，且不丢失数据。

3. 数据访问优化

3.1 连接池

系统通过 HikariCP 管理 MySQL 连接，所有 MySQL DAO 通过统一的 `MySQLDBUtil` 获取连接。这避免了每次操作都重新创建物理连接，并将连接的获取和释放交给成熟连接池管理。

当前使用 HikariCP 默认池参数。课程演示数据量和单机使用场景下无需增加额外调优配置。如需支持更高并发，应根据实际连接等待时间和数据库负载调整最大连接数。

HikariCP 的当前职责边界：

- 延迟创建和复用数据库连接。
- DAO 关闭 `Connection` 时将连接归还连接池。
- 避免业务代码自行维护连接列表和生命周期。
- 数据库连接参数由统一配置文件加载。

本项目不预先配置过大的连接池。Swing 客户端是单用户桌面程序，过多连接不能提高界面性能，反而会增加数据库端的连接成本。

3.2 PreparedStatement

MySQL DAO 使用 `PreparedStatement` 绑定参数。查询条件、更新值、主键和用户输入不直接拼接进 SQL，同时满足课程对 SQL 注入防护的要求。

`BaseDAO` 统一提供插入、更新、单行查询和列表查询功能，避免各 DAO 重复编写连接和结果集处理逻辑。

3.3 查询边界

- 库存、订单和用户列表只查询展示所需的业务数据。
- MongoDB 日志和评论查询带有数量上限，避免界面一次加载无界数据。
- 月度报表在 MySQL 存储过程中完成分组统计，界面不加载全部订单后再计算。
- MongoDB 热门批次、评分和审计数据通过聚合管道在数据库端统计。

3.4 资源释放

MySQL 数据访问使用 try-with-resources 管理 `Connection`、`PreparedStatement` 和 `ResultSet`。无论查询成功还是抛出异常，资源都会按相反顺序关闭。

事务方法在 `finally` 中恢复自动提交，避免连接归还连接池后 保留上一次事务的连接状态。

3.5 批量演示数据

MySQL 演示数据使用多值 `INSERT` 语句初始化，MongoDB 演示数据使用 `insertMany` 写入数组。这减少了初始化过程中的命令往返次数。

业务界面的单条新增和修改仍使用单条写入，因为用户操作没有可合并的大批量数据。

4. MySQL 索引检查

表	索引	主要用途	结论
users	uk_users_username	登录和用户名唯一校验	匹配高频等值查询
users	uk_users_email	邮箱唯一性	支持数据约束
categories	idx_categories_parent_id	查询子分类	匹配树形分类查询
items	idx_items_category_id	按分类查询库存	匹配分类筛选
items	idx_items_status	按可用状态查询	匹配状态筛选
items	idx_items_created_at	按时间查询	支持时间顺序访问
orders	idx_orders_user_id	查询用户申请	匹配普通用户的主要访问路径
orders	idx_orders_item_id	统计批次申请	支持关联和分组
orders	idx_orders_status	查询待审批记录	匹配管理员审批场景
orders	idx_orders_created_at	月度报表	支持时间范围查询
profiles	uk_profiles_user_id	按用户查询档案	同时保证一对一关系

索引与当前查询条件匹配。没有为低频字段增加额外索引，以免增加写入成本和维护负担。

4.1 关键查询执行计划

业务场景	命中索引	执行计划结果	结论
按用户名登录	uk_users_username	const，预计 1 行	唯一索引直接定位
按用户查询申请	idx_orders_user_id	ref，预计 5 行	先按用户缩小范围
查询待审批申请	idx_orders_status	ref，预计 8 行	状态索引生效
按分类查询库存	idx_items_category_id	ref，预计 7 行	分类索引生效
月度报表时间范围	idx_orders_created_at	range，预计 12 行	时间范围使用索引

4.2 EXPLAIN ANALYZE 结果

登录查询通过唯一索引在执行前定位一行数据。当前演示数据下，该查询的实际执行时间低于 0.001 毫秒。

按用户查询申请的执行路径：

```
Index lookup on orders using idx_orders_user_id
Sort: orders.created_at DESC
actual rows: 5
actual time: approximately 0.11 ms
```

查询先通过 `user_id` 索引定位 5 条数据，然后在小结果集上排序。

待审批申请查询直接使用 `idx_orders_status`：

```
Index lookup on orders using idx_orders_status
actual rows: 8
actual time: approximately 0.03 ms
```

月度报表使用 `created_at` 时间范围索引扫描 12 条申请，然后通过 `items` 和 `categories` 主键执行单行关联：

```
Index range scan on orders using idx_orders_created_at
Single-row index lookup on items using PRIMARY
Single-row index lookup on categories using PRIMARY
Aggregate using temporary table
actual time: approximately 1.94 ms
```

报表分组需要临时表，但时间范围筛选在聚合前已使用索引减少数据量。

4.3 额外排序的处理决策

“按用户查询申请并按创建时间倒序”使用 `idx_orders_user_id` 后仍执行一次额外排序。可选优化是增加复合索引：

```
CREATE INDEX idx_orders_user_created_at
ON orders (user_id, created_at DESC);
```

当前每个演示用户约 5 条申请，实际排序耗时约 0.02 毫秒。增加索引会为每次申请写入带来维护成本，因此当前不增加该索引。

当单个用户的申请数量达到数百条，或该查询在实际基准中成为慢查询时，再增加该复合索引。

4.4 索引成本控制

索引不是数量越多越好。每个额外索引都会：

- 占用磁盘和缓存空间。
- 增加插入、更新和删除时的维护工作。
- 增加数据库优化器的候选路径。

当前仅为实际查询条件、关联键、时间范围和唯一性要求建立索引。

5. MongoDB 索引检查

集合	索引	主要用途
action_logs	{ user_id: 1, created_at: -1 }	查询用户最近行为
action_logs	{ item_id: 1, created_at: -1 }	查询批次行为和热度
action_logs	{ action_type: 1, created_at: -1 }	按操作类型过滤
comments	{ item_id: 1, created_at: -1 }	查询批次评论
comments	{ user_id: 1, created_at: -1 }	查询用户评论
comments	{ rating: 1 }	评分统计
item_details	{ item_id: 1 }, unique	批次详情一对一关联
system_logs	{ user_id: 1, timestamp: -1 }	查询用户系统日志
system_logs	{ log_type: 1, timestamp: -1 }	按类型查询审计日志
system_logs	{ log_level: 1, timestamp: -1 }	按级别筛选异常

5.1 MongoDB 执行计划

业务查询	执行阶段	使用索引	检查键	检查文档	返回
用户最近行为	IXSCAN	idx_action_logs_user_created_at	10	10	10
批次最近评论	IXSCAN	idx_comments_item_created_at	1	1	1
最近登录日志	IXSCAN	idx_system_logs_type_timestamp	23	23	23
按批次查询详情	IXSCAN	uk_item_details_item_id	1	1	1

以上查询的索引键顺序同时覆盖等值过滤和时间倒序。执行计划中 没有出现全集合扫描，检查文档数与返回数一致。

5.2 聚合计算位置

操作热度、评分汇总和审计汇总在 MongoDB 聚合管道中完成。应用只接收聚合后的小型结果集，不将全部日志文档传输到 Java 内存再统计。

聚合管道的主要处理方式：

- 使用 `$match` 缩小需要统计的数据范围。
- 使用 `$group` 按批次、评分或日志类型分组。
- 使用 `$sort` 对统计结果排序，而不是对全部原始文档排序。
- 使用 `$limit` 限制热门结果数量。

5.3 日志和评论的读取上限

数据	单次读取上限	目的
库存批次评论	30	避免详情页一次加载全部历史评论
用户行为日志	由界面传入限制	只展示最近记录
系统日志	由界面传入限制	避免系统日志界面无界增长
热门批次	由服务传入限制	只返回展示需要的排名

5.4 日志生命周期

当前课程数据量不需要 TTL 索引。如系统长期运行，`action_logs` 和 `system_logs` 会持续增长。在出现明确的日志保留期要求后，可以选择：

- 为可自动过期的日志增加 TTL 索引。
- 将长期日志导出到归档存储。
- 保留审计汇总，删除已超过期限的低价值详细日志。

6. 事务和并发一致性

审批用血申请时，`OrderDAO.completeOrder` 执行以下操作：

1. 关闭自动提交。
2. 使用 `SELECT ... FOR UPDATE` 锁定申请和库存记录。
3. 检查申请状态、库存状态和可用数量。
4. 更新申请状态并扣减库存。
5. 全部成功后提交，异常时回滚。

删除已完成申请时，系统在同一事务中恢复库存并删除记录。MySQL 检查约束和触发器负责兜底校验数量与状态范围。

MySQL 与 MongoDB 之间没有分布式事务。库存主记录以 MySQL 为核心数据， MongoDB 保存可补写的详情和日志。如 MongoDB 在 MySQL 写入后失败， 可能出现详情暂缺， 但不影响库存和申请的核心状态。

6.1 锁定范围

`SELECT ... FOR UPDATE` 按申请主键定位一条申请， 并通过关联的 `item_id` 读取目标库存。事务不扫描或锁定全部库存， 不同批次的审批 可以独立处理。

6.2 事务内工作量

事务内只包含数据库查询、状态校验和两条更新语句。界面提示、 MongoDB 行为日志和其他非核心工作不放入 MySQL 事务， 减少持有行锁的时间。

6.3 事务失败路径

失败场景	处理结果
申请不存在	回滚并返回失败
申请已完成或已取消	回滚， 不重复扣减库存
库存已停用	回滚， 不更新申请
库存数量不足	回滚， 不出现负数库存
SQL 异常	捕获异常并显式回滚
删除已完成申请时恢复库存失败	删除和恢复同时回滚

6.4 数据库兜底校验

服务层在写入前提供可理解的业务提示， MySQL 表约束和触发器 负责防止非应用路径绕过校验。两层保护的职责不同：

- Java 校验负责及时反馈和业务语义。
- MySQL 检查约束保证数量和状态的最终合法性。
- MySQL 触发器防止已终结申请再次变更状态。

7. 压力测试

7.1 测试方法

压力测试使用 `SystemLogDA0StressTest` ， 创建 50 个并发工作线程， 向 MongoDB `system_logs` 集合写入 10000 条日志。

每次运行生成唯一 `run_id`。写入完成后按 `run_id` 统计文档数量，必须等于 10000。无论测试成功或失败，测试都尝试删除本次运行数据。

执行命令：

```
mvn -DstressTests=true -Dtest=SystemLogDAOStressTest test
```

7.2 实测结果

指标	结果
日志总数	10000 条
并发数	50
写入耗时	2.547 秒
平均吞吐量	3926.19 条/秒
数量校验	10000 条，与预期一致
Maven 结果	BUILD SUCCESS
数据清理	按 <code>run_id</code> 删除本次数据

结论：当前本地环境满足课程要求的 10000 条日志和 50 并发写入测试。该结果用于课程验收，不代表生产环境的容量承诺。

7.3 并发分配方式

测试创建固定 50 线程的执行器。第 `n` 个线程处理序号为 `n, n + 50, n + 100 ...` 的日志，每个线程约写入 200 条。

`CountDownLatch` 使 50 个工作线程在任务准备完成后同时开始，避免线程创建先后顺序对并发结果造成过大干扰。

7.4 正确性判定

压力测试不只记录耗时。通过以下步骤判定成功：

1. 所有 50 个工作任务在 2 分钟超时限制内完成。
2. 任意工作任务抛出异常时，测试失败。
3. 按唯一 `run_id` 统计实际写入数量。
4. 实际数量必须等于 10000。
5. 测试结束后删除本次 `run_id` 的数据。

因此“吞吐量较高但数据丢失”不会被记录为通过。

7.5 结果分析

分析项	结论
写入完整性	10000 条全部可按 <code>run_id</code> 查询，无数量丢失
并发可用性	50 个工作线程全部在限时内完成
吞吐量	本地单节点 MongoDB 达到 3926.19 条/秒
清理能力	测试数据不与演示日志混合，可定向删除
可重复性	测试命令、并发数、数据量和判定条件均固定

7.6 压力测试边界

- 测试对象是本地单节点 MongoDB，不包含跨网络延迟。
- 测试时间约 2.5 秒，不代表数小时稳定性。
- 每条日志尺寸较小，不代表含大型嵌套文档或文件的写入。
- 测试主要验证课程指定的并发写入，不是完整容量规划。

8. 安全检查清单

检查项	状态	证据或说明
密码不明文保存	通过	注册时使用 BCrypt 生成哈希，登录时使用 BCrypt 校验
SQL 不拼接用户输入	通过	DAO 通过 <code>PreparedStatement</code> 绑定参数
禁用账号拒绝登录	通过	登录服务检查 <code>status</code>
登录成功和失败可追溯	通过	写入 MongoDB <code>system_logs</code>
管理功能按角色隔离	通过	普通用户不显示用户、分类和系统日志管理入口
普通用户数据范围限制	通过	订单、档案、评论删除均校验当前用户
超级管理员保护	通过	禁止删除超级管理员，普通管理员不能修改用户权限
核心库存扣减事务	通过	行锁、显式提交和异常回滚
数据库约束	通过	主键、外键、唯一约束、检查约束和触发器
测试数据不使用真实隐私	通过	邮箱使用 <code>.test</code> ，证件号和地址为明确的测试值
敏感配置适合公开部署	待加固	<code>db.properties</code> 中包含本地演示密码，仅适用于课程环境
密码强度约束	待加固	当前仅检查密码非空，未限制长度和复杂度
登录失败限流	待加固	当前记录失败日志，未执行暂时锁定或限流

9. 已知边界与改进条件

9.1 本地凭据

`db.properties` 中的账号和密码用于课程演示容器。如应用部署到共享或公网环境，应改为由环境变量或密钥管理系统注入，并更换默认密码。

9.2 跨数据库写入

系统不引入分布式事务。如详情写入失败对业务造成不可接受的影响，可增加失败补写队列或定时对账，而不是扩大 MySQL 事务边界。

9.3 容量边界

当前压力测试聚焦 MongoDB 日志并发写入。如课程外需要评估长时间运行、大量用户或大数据量查询，应另行设计稳定性和查询基准测试。

9.4 界面线程边界

Swing 界面操作和数据加载主要由桌面应用触发。当前演示数据量下，查询能在可接受时间内返回。如单次查询耗时达到数百毫秒，需要将数据加载移到后台工作线程，避免阻塞 Swing 事件分发线程。

9.5 统计查询边界

月度报表的分组会使用临时表。当单月申请数据量显著增长时，应通过慢查询日志和 `EXPLAIN ANALYZE` 重新检查。可选升级路径包括：

- 按月建立预聚合汇总表。
- 在业务写入时异步更新报表读模型。
- 将大范围历史报表与实时业务查询分离。

这些方案仅在月度报表实际成为性能瓶颈时考虑。

10. 优化验证清单

检查项	验证方式	结果
MySQL 连接复用	HikariCP 统一提供 <code>Connection</code>	通过
JDBC 资源释放	<code>try-with-resources</code>	通过
SQL 参数绑定	检查 DAO 中的 <code>PreparedStatement</code>	通过
登录查询索引	<code>EXPLAIN</code> 命中 <code>uk_users_username</code>	通过
待审批查询索引	<code>EXPLAIN</code> 命中 <code>idx_orders_status</code>	通过

检查项	验证方式	结果
库存分类查询索引	EXPLAIN 命中 idx_items_category_id	通过
月度报表时间索引	EXPLAIN ANALYZE 命中 idx_orders_created_at	通过
MongoDB 用户行为索引	执行计划使用 IXSCAN	通过
MongoDB 批次评论索引	执行计划使用 IXSCAN	通过
MongoDB 登录日志索引	执行计划使用 IXSCAN	通过
MongoDB 批次详情唯一性	命中 uk_item_details_item_id	通过
库存审批并发一致性	行锁、事务、回归测试	通过
10000 条日志完整性	按 run_id 计数	通过
50 并发日志写入	独立压力测试	通过
压力测试数据清理	按 run_id 删除	通过
用户订单查询免排序	执行计划仍有小结果排序	按规模接受

11. 复测命令

常规测试：

```
mvn test
```

MongoDB 压力测试：

```
mvn -DstressTests=true -Dtest=SystemLogDAOStressTest test
```

MySQL 关键查询可在 hzcu_mysql 数据库中使用以下方式复测：

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE user_id = 1
ORDER BY created_at DESC;
```

```
EXPLAIN ANALYZE
SELECT c.name, COUNT(*) AS order_count
FROM orders o
JOIN items i ON i.item_id = o.item_id
JOIN categories c ON c.category_id = i.category_id
WHERE o.created_at >= '2026-07-01'
      AND o.created_at < '2026-08-01'
GROUP BY c.name;
```

MongoDB 用户行为查询可使用：

```
db.action_logs
  .find({ user_id: "1" })
  .sort({ created_at: -1 })
  .limit(50)
  .explain("executionStats");
```

12. 结论

系统已实现连接池、预编译语句、主要查询索引、MongoDB 组合索引、MySQL 显式事务和数据库约束。压力测试满足 10000 条日志、50 并发的课程验收规模，实测过程中无数量丢失。

安全基线满足课程项目要求。本地数据库凭据、密码强度和登录限流属于公开部署前的加固项，不影响当前课程演示和验收。